SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY
COMPUTER SCIENCE DEPARTMENT

SPRING 2025

Design and Analysis of Algorithms.
Assignment 2

Semester: 6th **Batch:** 2022F
Name: Muhammad Faisal **Section:** A
Roll #: 2022F-BCS-152

Q) Develop a menu-driven program in Python, C++, or C# that allows the user to choose from the following algorithms:
Select an option from the menu:
1) Linear Search
2) Binary Search
3) Bubble Sort
4) Selection Sort
5) Insertion Sort
6) Merge Sort
7) Naive Bayes Classifier
8) Rabin-Karp String Matching
9) Exit

C++ Code

```
#include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <algorithm>
using namespace std;

// 1. Linear Search
int linear_search(const vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target)
            return i;
    }
    return -1;
}

// 2. Binary Search
int binary_search(vector<int> arr, int target) {
    sort(arr.begin(), arr.end());
    int low = 0, high = arr.size() - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

```
// 3. Bubble Sort
void bubble_sort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1])
                swap(arr[j], arr[j + 1]);
}

// 4. Selection Sort
void selection_sort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[min_idx])
                min_idx = j;
        swap(arr[i], arr[min_idx]);
    }
}

// 5. Insertion Sort
void insertion_sort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// 6. Merge Sort
void merge(vector<int>& arr, int l, int m, int r) {
    vector<int> left(arr.begin() + l, arr.begin() + m + 1);
    vector<int> right(arr.begin() + m + 1, arr.begin() + r + 1);
    int i = 0, j = 0, k = l;

    while (i < left.size() && j < right.size())
        arr[k++] = (left[i] <= right[j]) ? left[i++] : right[j++];
    while (i < left.size()) arr[k++] = left[i++];
    while (j < right.size()) arr[k++] = right[j++];
}
void merge_sort(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        merge_sort(arr, l, m);
        merge_sort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
```

```
}

// 7. Naive Bayes
void naive_bayes_from_user() {
    int n, m;
    cout << "\n--- Naive Bayes (Euclidean Distance Approximation) ---\n";
    cout << "Enter number of training samples: ";
    cin >> n;
    cout << "Enter number of features per sample: ";
    cin >> m;

    vector<vector<double>> X(n, vector<double>(m));
    vector<int> y(n);

    cout << "Enter training data (features followed by label):\n";
    for (int i = 0; i < n; i++) {
        cout << "Sample " << i + 1 << ": ";
        for (int j = 0; j < m; j++) cin >> X[i][j];
        cin >> y[i];
    }

    int t;
    cout << "Enter number of test samples: ";
    cin >> t;

    for (int i = 0; i < t; i++) {
        vector<double> sample(m);
        cout << "Test Sample " << i + 1 << ": ";
        for (int j = 0; j < m; j++) cin >> sample[j];

        // Find nearest neighbor (approximate)
        double min_dist = 1e9;
        int pred_label = -1;
        for (int j = 0; j < n; j++) {
            double dist = 0;
            for (int k = 0; k < m; k++)
                dist += pow(sample[k] - X[j][k], 2);
            if (dist < min_dist) {
                min_dist = dist;
                pred_label = y[j];
            }
        }
        cout << "Predicted label: " << pred_label << "\n";
    }
}

// 8. Rabin-Karp
int rabin_karp(const string& text, const string& pattern, int prime = 101) {
    int n = text.length(), m = pattern.length(), d = 256;
    int p = 0, t = 0, h = 1;

    for (int i = 0; i < m - 1; i++)
        h = (h * d) % prime;
```

```
    for (int i = 0; i < m; i++) {
        p = (d * p + pattern[i]) % prime;
        t = (d * t + text[i]) % prime;
    }

    for (int i = 0; i <= n - m; i++) {
        if (p == t && text.substr(i, m) == pattern)
            return i;

        if (i < n - m) {
            t = (d * (t - text[i] * h) + text[i + m]) % prime;
            if (t < 0)
                t += prime;
        }
    }
    return -1;
}

// Utility function to print arrays
void print_array(const vector<int>& arr) {
    for (int val : arr) cout << val << " ";
    cout << endl;
}

// MAIN MENU
int main() {
    while (true) {
        cout << "\nSelect a choice from menu\n";
        cout << "1) Linear Search\n2) Binary Search\n3) Bubble Sort\n";
        cout << "4) Selection Sort\n5) Insertion Sort\n6) Merge Sort\n";
        cout << "7) Naive Bayes\n8) Rabin Karp\n9) Exit\n";

        int choice;
        cout << "Enter your choice (1-9): ";
        cin >> choice;

        if (choice >= 1 && choice <= 6) {
            int n;
            cout << "Enter number of elements: ";
            cin >> n;
            vector<int> arr(n);
            cout << "Enter elements: ";
            for (int i = 0; i < n; i++) cin >> arr[i];

            if (choice == 1 || choice == 2) {
                int target;
                cout << "Enter element to search: ";
                cin >> target;
                int index = (choice == 1) ? linear_search(arr, target)
                                          : binary_search(arr, target);
                if (index != -1)
                    cout << "Element found at index: " << index << "\n";
```

```
            else
                cout << "Element not found.\n";
        } else {
            switch (choice) {
                case 3: bubble_sort(arr); break;
                case 4: selection_sort(arr); break;
                case 5: insertion_sort(arr); break;
                case 6: merge_sort(arr, 0, arr.size() - 1); break;
            }
            cout << "Sorted array: ";
            print_array(arr);
        }

    } else if (choice == 7) {
        naive_bayes_from_user();

    } else if (choice == 8) {
        string text, pattern;
        cout << "Enter text: ";
        cin >> text;
        cout << "Enter pattern: ";
        cin >> pattern;
        int index = rabin_karp(text, pattern);
        if (index != -1)
            cout << "Pattern found at index: " << index << "\n";
        else
            cout << "Pattern not found.\n";

    } else if (choice == 9) {
        cout << "Exiting program.\n";
        break;

    } else {
        cout << "Invalid choice. Please select between 1 and 9.\n";
    }
  }

  return 0;
}
```

Output:

```
Select a choice from menu
1) Linear Search
2) Binary Search
3) Bubble Sort
4) Selection Sort
5) Insertion Sort
6) Merge Sort
7) Naive Bayes
8) Rabin Karp
9) Exit
Enter your choice (1-9):
```

```
Enter your choice (1-9): 1
Enter number of elements: 4
Enter elements: 1 3 4 5
Enter target to search: 3
Element found at index: 1
```

```
Enter your choice (1-9): 2
Enter number of elements: 3
Enter elements: 1 4 5
Enter target to search: 2
Element not found.
```

```
Enter your choice (1-9): 3
Enter number of elements: 4
Enter elements: 2 1 3 4 5
Sorted array: 1 2 3 4
```

```
Enter your choice (1-9): 4
Enter number of elements: 3
Enter elements: 3 2 1
Sorted array: 1 2 3
```

```
Enter your choice (1-9): 5
Enter number of elements: 4
Enter elements: 1 4 2 3
Sorted array: 1 2 3 4
```

```
Enter your choice (1-9): 6
Enter number of elements: 6
Enter elements: 9 5 7 2 5 1
Sorted array: 1 2 5 5 7 9
```

```
Enter your choice (1-9): 7

--- Naive Bayes (Euclidean Distance Approximation) ---
Enter number of training samples: 3
Enter number of features per sample: 2
Enter training data (features followed by label):
Sample 1: 2.2 3.0 1
Sample 2: 5.2 4.0 2
Sample 3: 3.3 6.1 3
Enter number of test samples: 2
Test Sample 1: 2.4 3.9
Predicted label: 1
Test Sample 2: 3.9 7.0
Predicted label: 3
```

```
Enter your choice (1-9): 8
Enter text: english
Enter pattern: li
Pattern found at index: 3
```